

Recommended Monitoring Stack

Prometheus, Alertmanager, and Grafana

kube-state-metrics and kubelet/cAdvisor metrics

Blackbox Exporter

Loki and Grafana Alloy

Service-specific Prometheus exporters

1. External Website Monitoring

Use Prometheus Blackbox Exporter to monitor the existing public WordPress homepage:

`https://example.com/`

Area	Monitor
Connectivity	DNS resolution, TCP connectivity, TLS negotiation, and redirects
Availability	HTTP response status and expected homepage content
Performance	Total response time and individual DNS, connect, TLS, and transfer phases
Certificate	TLS certificate expiration

The probe can require HTTP 200 and search for a stable page title or another known string. Blackbox Exporter should preferably run outside the WordPress Kubernetes cluster so it can detect complete cluster, load balancer, DNS, certificate, or network failures.

Availability alert

```
probe_success{job="wordpress-public"} == 0
for: 2m
```

Certificate expiration

```
(probe_ssl_earliest_cert_expiry - time()) < 14 * 86400
```

- Warning: fewer than 14 days remaining
- Critical: fewer than 3 days remaining

2. Kubernetes Workload Monitoring

Use kube-state-metrics and kubelet metrics to monitor Kubernetes object state and workload availability.

Area	Monitor
Workload state	Pods not ready, Pending pods, CrashLoopBackOff, and replica mismatch
Container lifecycle	Container restarts and OOMKilled terminations
Storage	PersistentVolumeClaim availability and persistent volume capacity
Resources	Container CPU throttling and memory usage

Recommended alerts

- WordPress Deployment has unavailable replicas for more than 5 minutes
- MySQL StatefulSet has unavailable replicas
- A container restarted more than three times within 15 minutes
- A container was terminated because of OOM
- Persistent volume has less than 15% free space or is predicted to fill within 24 hours
- A Pod remains not ready for more than 5 minutes

Important: Kubernetes workload status does not confirm that the public website is accessible. Combine it with Blackbox monitoring.

3. Container Resource Monitoring

Use kubelet/cAdvisor metrics to monitor WordPress, NGINX, PHP-FPM, Varnish, MySQL, Memcached, and the ingress controller.

- CPU usage and CPU throttling
- Memory working set and memory limits
- Container network traffic
- Container filesystem usage
- Container start time

4. Load Balancer and Ingress Monitoring

The selected Kubernetes load balancer and ingress implementation should provide the following metrics:

- Request rate and HTTP 4xx/5xx responses
- Healthy and unhealthy backend targets
- Request duration, including p95 and p99 latency
- Active and rejected connections
- TLS and connection errors

For EKS with an AWS Application Load Balancer, these metrics can be collected from AWS CloudWatch. If an ingress controller is used, enable its native Prometheus metrics.

Recommended alerts

- No healthy backend targets
- HTTP 5xx rate above 2% for 5 minutes
- p95 response time above 2 seconds for 10 minutes
- Increasing TLS or connection errors
- Ingress controller unavailable

Alert quality: Include a minimum request volume in error-rate alerts so one failed request does not produce a misleading 100% error rate.

5. MySQL Monitoring

Use mysqld_exporter with a dedicated read-only monitoring account.

Area	Monitor
Availability	Exporter availability, MySQL availability, and uptime
Connections	Active connections, maximum connection usage, and threads running

Area	Monitor
Queries	Query rate, slow queries, deadlocks, lock waits, and temporary tables written to disk
InnoDB	Buffer pool use and related storage behavior
Storage/replication	Database storage and replication lag when replication is introduced

Recommended alerts

- MySQL exporter unavailable
- Connections above 80% of the configured limit
- Deadlocks detected or a sustained increase in slow queries
- Database storage approaching capacity
- Replication lag above the accepted threshold

6. Memcached Monitoring

Use a maintained Memcached Prometheus exporter.

Area	Monitor
Availability	Exporter and Memcached availability
Capacity	Current connections, memory usage, and configured memory limit
Cache efficiency	Evictions, cache hits, cache misses, hit ratio, and command rate

Recommended alerts

- Memcached exporter unavailable
- Memory usage above 85%
- Sustained cache evictions or connection exhaustion
- Significant reduction in cache hit ratio

Cache hit ratio thresholds should be introduced only after establishing a normal production baseline.

7. Varnish Monitoring

Use the standard varnishstat counters. A community Varnish Prometheus exporter can expose these counters to Prometheus.

- Cache hits, misses, and cache hit ratio
- Backend health, connection failures, and fetch failures
- Cache storage usage
- Thread pool usage and queued requests
- Dropped requests and ban operations

Version control: Commonly available Varnish exporters are community-maintained. Pin the selected image to an exact version and review it before production use.

8. NGINX Monitoring

NGINX runs inside the WordPress container and communicates with PHP-FPM. Enable the standard NGINX stub_status module and use NGINX Prometheus Exporter.

Recommended Kubernetes deployment

- NGINX remains inside the WordPress container
- NGINX Prometheus Exporter runs as a sidecar container in the same Pod

- `stub_status` is accessible only from within the Pod
- Prometheus collects exporter metrics through a `ServiceMonitor`

Metrics

- Active, accepted, and handled connections
- Total request count
- Reading, writing, and waiting connections

Limitation: Standard NGINX `stub_status` does not provide HTTP status-code or request-latency metrics. Collect those from the load balancer, ingress controller, access logs, or Blackbox Exporter.

9. WordPress and PHP-FPM Monitoring

WordPress does not provide a standard Prometheus metrics endpoint. Monitor availability through the existing homepage, platform metrics, and logs.

- External HTTP probes against the homepage
- Load balancer and ingress metrics
- NGINX metrics
- PHP-FPM and WordPress logs
- Kubernetes Pod and container metrics

PHP-FPM can optionally expose its standard status page internally. A reviewed PHP-FPM exporter can then provide active and idle worker counts, listen queue depth, max-children events, slow requests, and worker failures.

Security: The PHP-FPM status page must remain accessible only inside the Pod or Kubernetes cluster.

10. Centralized Logging

Store logs in Grafana Loki and display them in Grafana. Promtail should not be selected for a new deployment: it reached end of life on March 2, 2026. Use Grafana Alloy instead.

Collection

Run Grafana Alloy as a Kubernetes DaemonSet and collect Kubernetes Pod logs, load balancer or ingress logs, Varnish logs, NGINX access and error logs, PHP-FPM logs, WordPress logs, MySQL error and slow-query logs, and Memcached logs.

Recommended Loki labels

- cluster, namespace, application
- container, pod, environment

Request IDs, user IDs, URLs, and IP addresses should remain in the log message rather than being used as Loki labels.

Recommended log-based alerts

- Rapid increase in HTTP 5xx responses
- PHP fatal errors or WordPress database connection errors
- MySQL crashes, corruption messages, or sustained slow-query increases
- NGINX upstream connection failures
- Varnish backend fetch failures
- Repeated container startup failures

11. Alerting and Dashboards

Prometheus should evaluate metric-based alert rules. Alertmanager should handle notification routing, grouping, inhibition, silences, and delivery to email, Slack, Teams, or PagerDuty.

Recommended Grafana dashboards

1. Public website availability and TLS
2. Kubernetes workload status
3. Container resource usage
4. Load balancer and ingress traffic
5. MySQL
6. Memcached
7. Varnish
8. NGINX and PHP-FPM
9. Centralized application logs

Incident grouping: A MySQL failure may generate WordPress errors, unavailable Pods, and HTTP 5xx responses. Group related signals into one incident where possible.

Recommended Minimum Implementation

1. Prometheus, Alertmanager, and Grafana
2. kube-state-metrics
3. kubelet/cAdvisor metrics
4. Blackbox Exporter checking the public WordPress homepage
5. Load balancer or ingress metrics
6. mysqld_exporter
7. Memcached exporter
8. Varnish exporter based on varnishstat
9. NGINX Prometheus Exporter as a WordPress Pod sidecar
10. Loki
11. Grafana Alloy for Kubernetes log collection

Private access only: Exporter ports and metrics endpoints should be accessible only through Kubernetes ClusterIP services, ServiceMonitor resources, and appropriate NetworkPolicy rules.

References

1. Prometheus Blackbox Exporter guide
<https://prometheus.io/docs/guides/multi-target-exporter/>
2. kube-prometheus-stack
<https://github.com/prometheus-community/helm-charts/tree/main/charts/kube-prometheus-stack>
3. MySQL Prometheus Exporter
https://github.com/prometheus/mysqld_exporter
4. Prometheus exporter catalog
<https://prometheus.io/docs/instrumenting/exporters/>
5. Varnish statistics
<https://varnish-cache.org/docs/7.6/users-guide/operation-statistics.html>
6. NGINX Prometheus Exporter
<https://github.com/nginx/nginx-prometheus-exporter>
7. Grafana Alloy
<https://grafana.com/docs/alloy/latest/monitor/>
8. Promtail end-of-life notice
<https://grafana.com/docs/loki/latest/send-data/promtail/>